

Padrões de Arquitetura + Arquitetura REST

Da rede à API: como os dados viajam até virar um endpoint REST

Conceito → protocolos de rede e o HTTP por dentro • Prática → construindo uma API REST com Spring Web

O que vamos ver hoje

- Parte 1 — Padrões de Arquitetura: os protocolos por trás da Web (TCP/IP, DNS, HTTP) e como o HTTP evoluiu.
- Parte 2 — Arquitetura REST na prática: construindo uma API REST de verdade com Spring Web.
- Vamos sair hoje com o primeiro CRUD rodando: Controller, Service, Repository e DTO.

1. Padrões de Arquitetura

TCP/IP, DNS, HTTP por dentro e HTTPS

2. Arquitetura REST

Spring Web: controllers, services, DTOs

PARTE 1

Padrões de Arquitetura

Os protocolos que sustentam toda comunicação na Web, camada por camada

De onde viemos

- Na Aula 06, vimos o que é um Web Service e como o HTTP estrutura requisição e resposta.
- Vimos métodos, headers, status codes e como JSON padroniza os dados trocados.
- Hoje vamos abrir uma camada abaixo do HTTP — os protocolos que fazem a mensagem realmente chegar ao destino — e depois subir para construir uma API REST de verdade.

Os protocolos por trás de cada requisição

- Quando o navegador (ou o Postman) faz uma requisição, vários protocolos trabalham em conjunto, em camadas.
- TCP e IP cuidam do endereçamento e da transmissão dos pacotes de dados pela rede.
- DNS traduz o nome de domínio que digitamos em um endereço IP que a rede entende.
- HTTP e HTTPS cuidam do tráfego de mensagens entre as aplicações — é a camada que vimos na Aula 06.

TCP/IP

Endereçamento e transmissão dos pacotes de dados.

DNS

Tradução de domínio (nome) para endereço IP.

HTTP/HTTPS

Tráfego de mensagens entre as aplicações.

A Web em camadas

- Cada camada resolve um problema diferente e depende da camada de baixo para funcionar.
- HTML, CSS, JavaScript e as Web APIs do navegador ficam no topo — o que chamamos de 'a Web'.
- HTTP é a camada de aplicação que carrega essas mensagens.
- TCP (ou o par TLS+TCP, no HTTPS) garante a entrega; IP cuida do roteamento; DNS resolve nomes em paralelo.

HTML / CSS / JS / Web APIs

O que rotulamos como 'a Web'

HTTP

Camada de aplicação — mensagens

TLS (quando HTTPS)

Criptografia da conexão

TCP

Entrega, ordem e integridade dos pacotes

DNS / IP

Tradução de nomes e roteamento

Cabeçalhos HTTP comuns (parte 1)

Cabeçalho	Descrição	Exemplo
Host	Nome do servidor de destino	example.com
Content-Length	Tamanho do corpo da mensagem, em bytes	73
Content-Type	Tipo do corpo da mensagem	text/html
User-Agent	Identificação do cliente que fez a chamada	Mozilla/5.0 (Windows...)
Authorization	Credenciais de autenticação	Bearer <token>

Cabeçalhos HTTP comuns (parte 2)

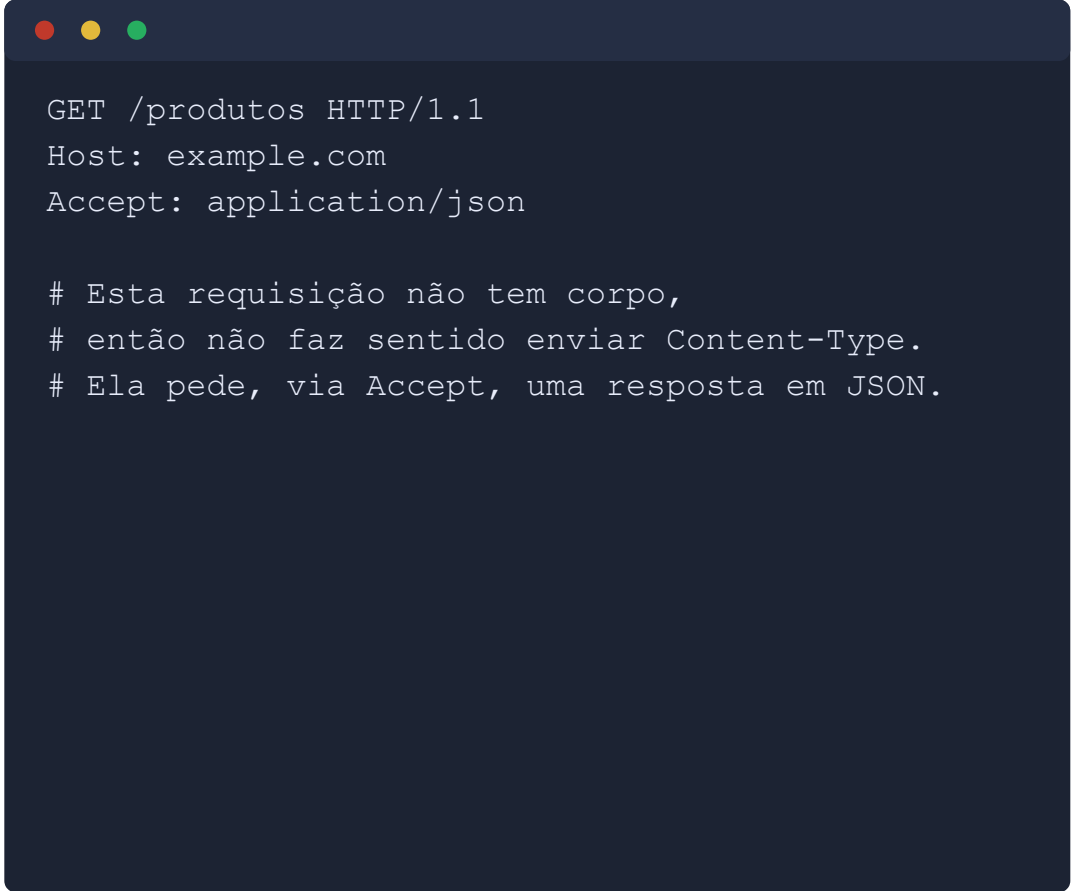
Cabeçalho	Descrição	Exemplo
Accept	Tipos de conteúdo aceitos pelo cliente	text/html
Cookie	Dados de sessão do cliente	sessionId=abc123; userId=1
Connection	Controle de conexão	keep-alive ou close
Cache-Control	Controle de cache	no-cache, no-store, must-revalidate
Last-Modified	Data e hora da última modificação do recurso	Wed, 21 Oct 2015 07:28:00 GMT

Content-Type: os formatos mais comuns

Tipo	Content-Type	Uso comum
Texto genérico	text/plain	Texto simples, sem formatação
HTML	text/html	Páginas web
JSON	application/json	APIs REST — o formato que mais usaremos
Formulário	multipart/form-data	Envio de formulários com arquivos
Binário	application/octet-stream	Arquivos genéricos, sem formato conhecido
Imagem / Áudio / Vídeo	image/png, audio/mpeg, video/mp4	Arquivos de mídia

Content-Type x Accept: qual a diferença?

- Content-Type descreve o formato do corpo que está sendo enviado nesta mensagem.
- Accept descreve os formatos que o cliente está disposto a aceitar na resposta.
- Ou seja: Content-Type fala do que eu estou mandando; Accept fala do que eu quero receber de volta.
- Em uma API REST, é comum enviar e receber sempre em application/json — mas a distinção importa quando uma API suporta mais de um formato.



```
GET /produtos HTTP/1.1
Host: example.com
Accept: application/json

# Esta requisição não tem corpo,
# então não faz sentido enviar Content-Type.
# Ela pede, via Accept, uma resposta em JSON.
```

401 x 403: dois erros que se confundem

- 401 Unauthorized: o cliente não se autenticou corretamente — o servidor não sabe (ou não confirma) quem está fazendo a requisição.
- 403 Forbidden: o servidor sabe exatamente quem é o cliente, mas ele não tem permissão para aquela ação.
- Um jeito de lembrar: 401 é 'quem é você?'; 403 é 'eu sei quem você é, mas não pode'.

401 Unauthorized

Ex.: token inválido ou ausente. Faltou provar quem você é.

403 Forbidden

Ex.: usuário comum tentando acessar uma rota administrativa.

Cache HTTP: evitando trabalho repetido

- Cabeçalhos de cache ajudam a evitar transferências desnecessárias entre cliente e servidor.
- Cache-Control define por quanto tempo (e sob quais regras) uma resposta pode ser reaproveitada sem nova requisição.
- ETag identifica uma versão específica do recurso — se o recurso não mudou, o servidor pode responder 'sem mudanças' em vez de reenviar tudo.

```
HTTP/1.1 200 OK
Cache-Control: max-age=3600
ETag: "produto-10-v3"

# max-age=3600: o cliente pode reaproveitar
# esta resposta por até 3600 segundos (1h)
# sem perguntar de novo ao servidor.
```

HTTPS: HTTP com uma camada de segurança

- HTTPS é uma extensão do protocolo HTTP, usando uma camada de segurança chamada TLS (Transport Layer Security).
- A negociação da criptografia usa chaves públicas e acontece já na fase de conexão, antes da primeira requisição HTTP.
- Roda por padrão na porta 443, diferente da porta 80 do HTTP puro.
- TLS é a versão mais recente do antigo SSL (Secure Sockets Layer) — o termo 'SSL' ainda aparece bastante, mesmo referindo-se ao TLS atual.

HTTP

Porta 80. Sem criptografia — dados trafegam em texto plano.

HTTPS

Porta 443. HTTP + TLS — dados criptografados no trajeto.

401, 403 ou cache: qual se aplica?

Objetivo: para cada cenário abaixo, identifique o status code (ou cabeçalho) correto e justifique em uma frase.

- Um usuário tenta acessar a API sem enviar nenhum token de autenticação.
- Um usuário autenticado (comum) tenta acessar uma rota exclusiva de administradores.
- Uma resposta que não muda com frequência deveria evitar ser buscada de novo a cada poucos segundos — qual cabeçalho resolve isso?
- Um cliente reenvia a mesma requisição DELETE duas vezes seguidas, por conta da internet ter oscilado — isso deveria ser um problema?

PARTE 2

Arquitetura REST com Spring Web

Colocando tudo isso para funcionar: sua primeira API REST em Java

O que é Spring?

- Spring é um ecossistema de frameworks para desenvolvimento em Java.
- Seu foco é produtividade, organização e baixo acoplamento entre as partes do sistema.
- É muito usado para APIs REST, sistemas corporativos e microsserviços — exatamente o que estamos construindo no curso.
- Dentro desse ecossistema, o módulo Spring Web facilita especificamente a criação de aplicações HTTP.

Spring Framework x Spring Boot

- Spring Framework fornece os módulos e a base de todo o ecossistema.
- Spring Boot simplifica a configuração e a inicialização do projeto por cima dessa base.
- Com Spring Boot, o próprio servidor web (Tomcat, embutido) já vem pronto — sem precisar instalar nada à parte.
- O objetivo do Spring Boot é reduzir configurações manuais e acelerar o desenvolvimento — é o que vamos usar no curso.

Spring Framework

A base: módulos, injeção de dependência, infraestrutura.

Spring Boot

Configuração automática + servidor embutido, pronto para rodar.

Por que usar Spring Web?

- Criação de rotas HTTP usando anotações — sem precisar escrever o parsing do protocolo na mão.
- Conversão automática entre JSON e objetos Java, nos dois sentidos.
- Integração pronta com validação, injeção de dependência e tratamento de erros.
- Estrutura adequada para separar controller, service e repository, cada um com sua responsabilidade.

Framework x Boot x Web, em uma frase:

- Framework = a base;
- Boot = o jeito fácil de ligar a base (autoconfiguração + Servidor Tomcat embutido, é o que essa anotação ativa);
- Web = o módulo da base focado em HTTP (Controller, @GetMapping...), que o Boot já traz configurado.

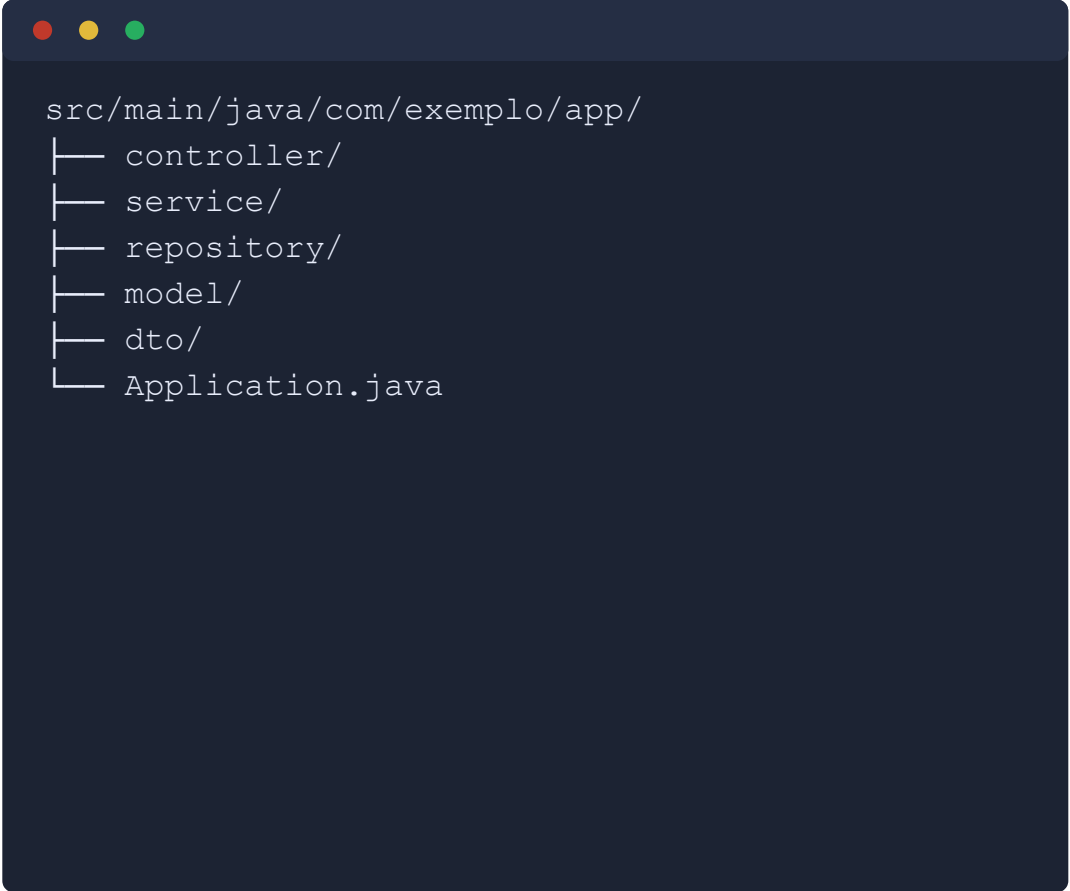
Dependências comuns de um projeto Spring Boot

Dependência	Para que serve
spring-boot-starter-web	Cria APIs HTTP/REST usando Spring MVC, com servidor embutido
spring-boot-starter-validation	Habilita anotações de validação (@NotBlank, @Positive...)
spring-boot-starter-data-jpa	Persistência de dados em banco SQL, usando JPA e Hibernate
spring-boot-devtools	Recarrega a aplicação automaticamente durante o desenvolvimento

Essas dependências aparecem já na criação do projeto, no Spring Initializr.

Estrutura básica de um projeto Spring

- Cada pacote representa uma camada, com uma responsabilidade bem definida.
- controller: recebe requisições HTTP.
- service: concentra as regras de negócio.
- repository: acessa o banco de dados.
- model (ou entity) e dto: representam os dados da aplicação e o formato de entrada/saída da API.



```
src/main/java/com/exemplo/app/  
├── controller/  
├── service/  
├── repository/  
├── model/  
├── dto/  
└── Application.java
```

O diagrama mostra a estrutura de pastas de um projeto Spring. No topo, há uma barra decorativa com três pontos coloridos (vermelho, amarelo, verde). Abaixo, o caminho de pastas é exibido em um formato de árvore: 'src/main/java/com/exemplo/app/'. Sob este caminho, há cinco pastas: 'controller/', 'service/', 'repository/', 'model/' e 'dto/'. Abaixo das pastas, há um arquivo 'Application.java'.

Essa organização em camadas facilita manutenção, testes e evolução da aplicação.

A classe principal

- @SpringBootApplication ativa as configurações automáticas do projeto (é um atalho para três outras anotações).
- SpringApplication.run(...) inicializa o contexto do Spring e sobe a aplicação (o servidor embutido).
- É essa classe que você executa (o 'play' da sua IDE) para ligar a API.

```
@SpringBootApplication
public class Application {

    public static void main(String[] args) {

        SpringApplication.run(Application.class, args);
    }
}
```

Controllers: a porta de entrada da API

- Controllers são as classes responsáveis por receber requisições HTTP e devolver respostas.
- `@RestController` indica que esta classe é um controller de API REST (as respostas viram JSON automaticamente).
- `@RequestMapping` define o caminho base de todas as rotas dessa classe.

```
@RestController
@RequestMapping("/produtos")
public class ProdutoController {

    // os endpoints entram aqui

}
```

Mapeando endpoints

- @GetMapping atende requisições GET — normalmente para listar ou buscar dados.
- @PostMapping atende requisições POST — normalmente para criar um novo recurso.
- @RequestBody converte o JSON recebido no corpo da requisição em um objeto Java.

```
@GetMapping
public List<Produto> listar() {
    return service.listar();
}

@PostMapping
public Produto criar(@RequestBody Produto
produto) {
    return service.criar(produto);
}
```

Lendo parâmetros da requisição

- `@PathVariable` lê partes da própria URL — ex.: o id dentro de `/produtos/10`.
- `@RequestParam` lê parâmetros da query string — ex.: o `?nome=...` depois do caminho.
- Ambos são convertidos automaticamente para o tipo Java declarado (`Long`, `String`...).

```
@GetMapping("/{id}")
public Produto buscarPorId(@PathVariable Long id) {
    return service.buscarPorId(id);
}

@GetMapping("/buscar")
public List<Produto> buscarPorNome(
    @RequestParam String nome) {
    return service.buscarPorNome(nome);
}
```

Controlando a resposta com ResponseEntity

- ResponseEntity dá controle total sobre status, headers e corpo da resposta.
- Muito usado quando o status de sucesso não é simplesmente 200 — como 204 No Content em uma remoção.
- Combinações comuns: 200 OK, 201 Created, 204 No Content, 404 Not Found.

```
@DeleteMapping("/{id}")
public ResponseEntity<Void>
remove(@PathVariable Long id) {
    service.remove(id);
    return ResponseEntity.noContent().build();
}
```

Injeção de dependência

- No Spring, objetos podem ser gerenciados pelo container e injetados automaticamente, em vez de você criá-los com new.
- @Service marca uma classe como responsável por regras de negócio.
- O Spring identifica que ProdutoController precisa de um ProdutoService e entrega essa dependência sozinho, pelo construtor.

```
@Service
public class ProdutoService {

    private final ProdutoRepository repository;

    public ProdutoService(ProdutoRepository
repository) {
        this.repository = repository;
    }
}
```

As camadas de uma aplicação Spring

Camada	Responsabilidade
Controller	Recebe a requisição HTTP e monta a resposta
Service	Concentra as regras de negócio da aplicação
Repository	Acessa o banco de dados (consulta e persiste)
Model / Entity	Representa os dados da aplicação
DTO	Estrutura os dados de entrada e saída da API

Validação de dados com DTO

- @Valid ativa a validação do objeto recebido, antes mesmo de ele chegar na regra de negócio.
- Anotações como @NotBlank e @Positive evitam que dados inválidos cheguem até o service.
- Usar um DTO (em vez da entidade direto) evita expor no contrato da API campos que só existem no banco de dados.

```
public class ProdutoDTO {  
    @NotBlank  
    private String nome;  
  
    @Positive  
    private BigDecimal preco;  
}  
  
public Produto criar(@Valid @RequestBody ProdutoDTO  
dto) {  
    return service.criar(dto);  
}
```

JSON automaticamente, sem esforço extra

- Quando um método retorna um objeto Java em um `@RestController`, o Spring normalmente serializa a resposta em JSON sozinho.
- É a mesma conversão que vimos com o Jackson na Aula 06 — só que o Spring já faz por trás dos panos, sem você chamar o `ObjectMapper` manualmente.

```
@GetMapping("/{id}")
public ProdutoDTO detalhar(@PathVariable Long id) {
    return service.detalhar(id);
}

// Resposta esperada:
// {
//   "id": 10,
//   "nome": "Teclado",
//   "preco": 199.90
// }
```

Tratamento de exceções, centralizado

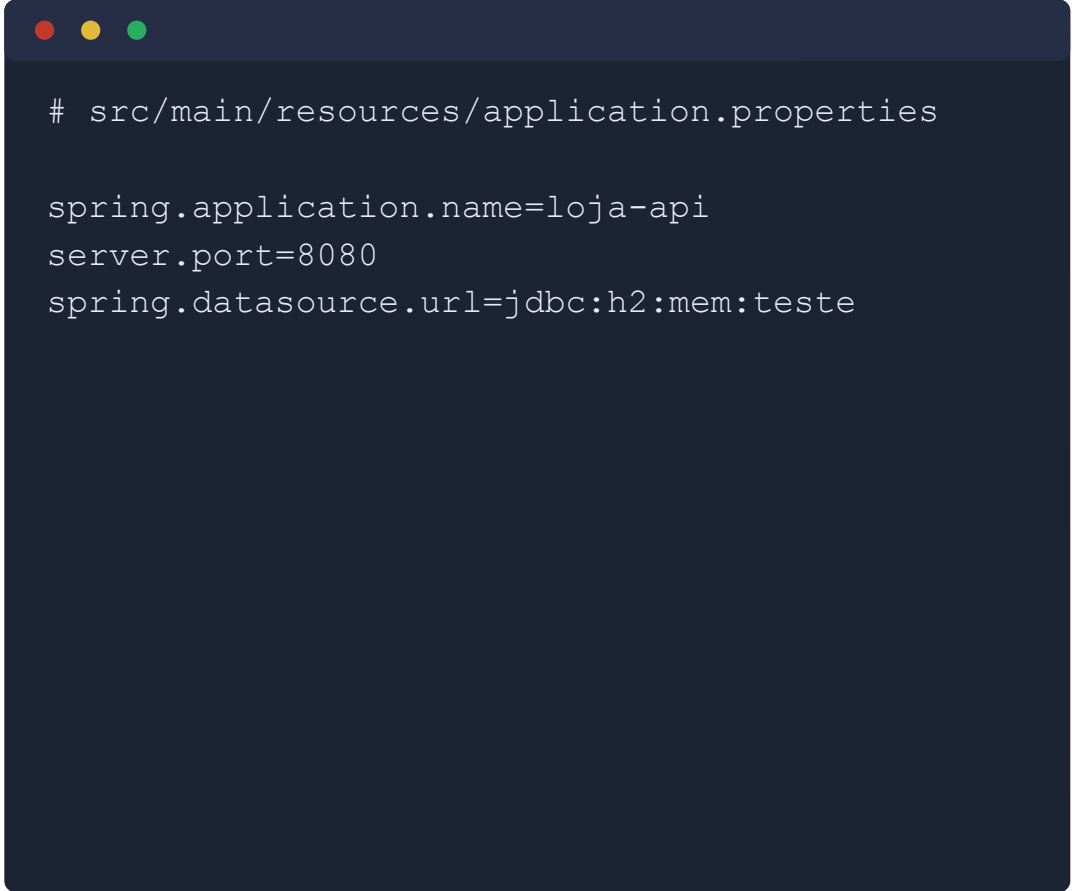
- `@RestControllerAdvice` centraliza o tratamento de erros para todos os controllers da aplicação.
- `@ExceptionHandler` indica qual tipo de exceção aquele método trata.
- Isso evita repetir blocos try/catch em cada controller, um por um.

```
@RestControllerAdvice
public class ApiExceptionHandler {

    @ExceptionHandler(ProdutoNaoEncontradoException.class)
    public ResponseEntity<String> tratarNaoEncontrado()
    {
        return ResponseEntity.status(404)
            .body("Produto não encontrado");
    }
}
```

Configurando a aplicação

- application.properties (ou application.yml) guarda as configurações da aplicação.
- spring.application.name identifica o projeto; server.port define em qual porta ele sobe.
- spring.datasource.url aponta para o banco de dados que a aplicação vai usar.



```
# src/main/resources/application.properties

spring.application.name=loja-api
server.port=8080
spring.datasource.url=jdbc:h2:mem:teste
```

O fluxo completo de uma requisição

- 1. O cliente envia uma requisição HTTP (ex.: pelo Postman ou por um app).
- 2. O Controller recebe e identifica a rota correspondente.
- 3. O Service aplica as regras de negócio sobre os dados recebidos.
- 4. O Repository consulta ou persiste os dados no banco.
- 5. A resposta volta ao cliente como JSON, junto com um status HTTP.

Esse é o mesmo fluxo cliente-servidor da Aula 01 — só que agora, com nome para cada camada.

Fluxo de uma requisição — POST /tarefas

Do Tomcat ao TarefaRepository, e de volta como JSON



Juntando tudo: uma rota REST completa

- Esse é o mesmo formato de contrato de API que vimos na Aula 06 — agora, implementado de verdade com Spring Web.
- POST /produtos cria um recurso nesse contrato: recebe o corpo, devolve 201 Created.

```
# Requisição
POST /produtos HTTP/1.1
Content-Type: application/json

{ "nome": "Mouse", "preco": 89.90 }

# Resposta
HTTP/1.1 201 Created
Content-Type: application/json

{ "id": 12, "nome": "Mouse", "preco": 89.90 }
```

Boas práticas ao construir uma API REST

Prática	Por quê
Separar responsabilidade entre camadas	Facilita manutenção, testes e leitura do código
Evitar regra de negócio no controller	Controller só recebe e devolve — a lógica fica no service
Usar DTOs em vez de expor entidades	Evita acoplar o contrato da API à estrutura do banco
Validar entradas recebidas pela API	Evita dados inválidos chegando até a regra de negócio
Retornar códigos HTTP coerentes	O cliente consegue reagir corretamente sem ler o corpo

Construindo um endpoint de listagem e criação

Objetivo: implementar os dois primeiros endpoints do recurso /tarefas, usando as camadas Controller e Service.

- Crie TarefaController com @RestController e @RequestMapping("/tarefas").
- Implemente um @GetMapping que devolve a lista de tarefas, chamando um TarefaService.
- Implemente um @PostMapping que recebe uma TarefaDTO (@RequestBody) e devolve a tarefa criada.
- Teste os dois endpoints com curl ou Postman antes de seguir para o próximo exercício.

Completando o CRUD de /tarefas

Objetivo: finalizar o contrato de API de tarefas desenhado na Aula 06, agora com Spring Web de verdade.

- Adicione `@GetMapping("/{id}")` e `@DeleteMapping("/{id}")` ao `TarefaController`.
- Crie `TarefaDTO` com validação (`@NotBlank` no título) e use `@Valid` no endpoint de criação.
- No DELETE, devolva 204 No Content com `ResponseEntity`, e trate o caso de tarefa inexistente com 404.
- Crie um `@RestControllerAdvice` para centralizar o tratamento de `TarefaNaoEncontradaException`.

```
@DeleteMapping("/{id}")
public ResponseEntity<Void>
remove(@PathVariable Long id) {
    service.remove(id);
    return ResponseEntity.noContent().build();
}
```

RECAPITULANDO

O que levamos para casa

- A Web é feita de camadas de protocolos: TCP/IP fazem a entrega, DNS traduz nomes, HTTP carrega as mensagens.
- HTTP evoluiu (1.0 → 1.1 → 2 → 3), mas a essência de requisição/resposta continua a mesma; HTTPS adiciona TLS por cima.
- Spring Boot + Spring Web reduzem a distância entre 'entender REST' e 'ter uma API rodando'.
- Controller, Service, Repository e DTO — cada camada com uma responsabilidade — é o padrão que vamos repetir o semestre inteiro.

Dever de casa

Finalize os 3 exercícios (check de status codes, endpoints de listagem/criação e o CRUD completo de /tarefas) antes da próxima aula.

Para ir além

- Documentação oficial do Spring Boot — guias e referência completa do framework.
- MDN Web Docs — HTTP: aprofunda protocolos, versões do HTTP e cabeçalhos.
- Spring Initializr (start.spring.io) — gera a estrutura inicial de um projeto Spring Boot.
- Repositório da disciplina: github.com/placidodaniel/backend-aulas-satc